Artificial Intelligence

Course Code: CS-202

Topic: Search Algorithms

Instructor: Habiba Arshad

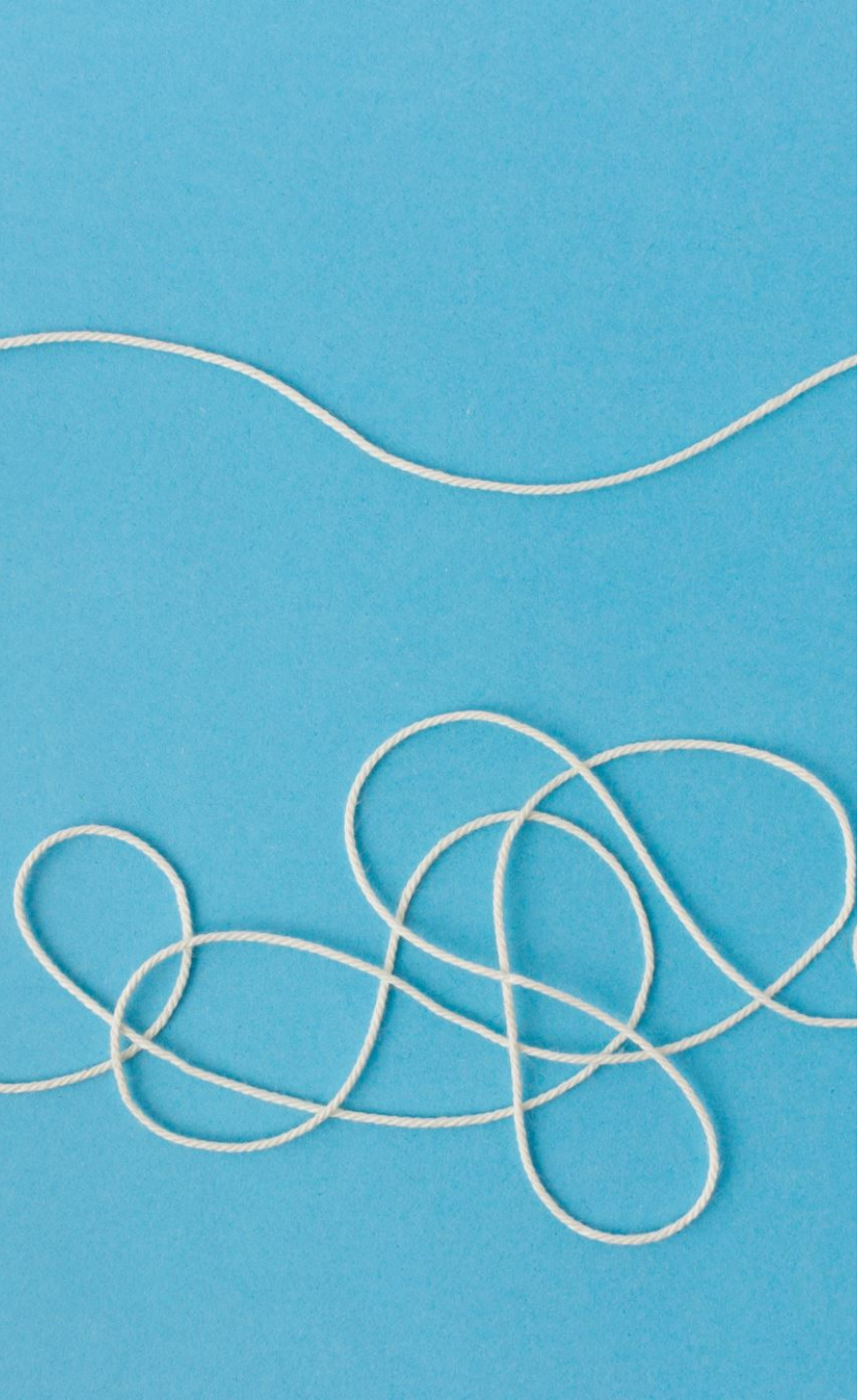
Week-4

This week lecture will cover the following CLO:

- **CLO 2:** . Implement artificial intelligence techniques and case studies

Uninformed Search Algorithm

- Blind Search Algorithm and have no additional information about the goal location beyond the problem definition



Uninformed Search Algorithms

Breadth-First Search (BFS)

- Explores all nodes at the present depth level before moving on to the next level.
- Guarantees finding the shortest path solution.

Depth-First Search (DFS)

- Explores as far as possible along each branch before backtracking.
- May not find the shortest path and can get stuck in loops.

Uniform Cost Search

- Expands the node with the lowest cost, ensuring an optimal solution if costs are non-negative.

Breadth-First Search (BFS)

- The algorithm starts from a given source and explores all reachable vertices from the given source. We begin with the given source (in tree, we begin with root) and traverse vertices level by level using a queue data structure.

Working:

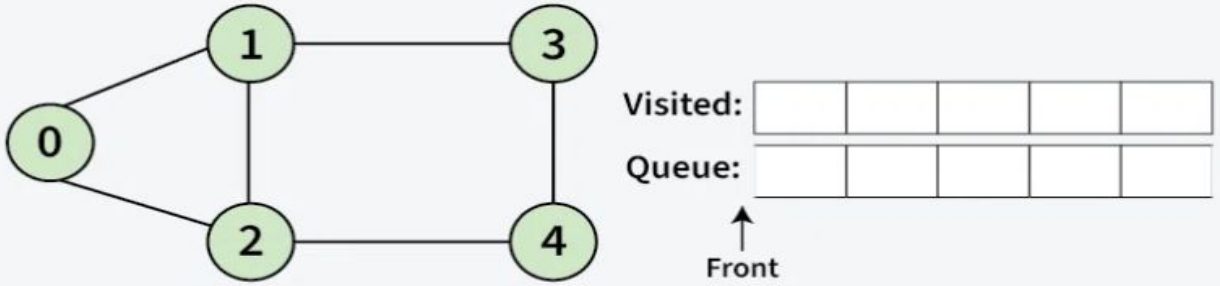
- **Initialization:** Enqueue the given source vertex into a queue and mark it as visited.
- **Exploration:** While the queue is not empty:
  1. Dequeue a node from the queue and visit it (e.g., print its value).
  2. For each unvisited neighbour of the dequeued node:
     - Enqueue the neighbours into the queue.
     - Mark the neighbour as visited.

Termination: Repeat step 2 until the queue is empty.

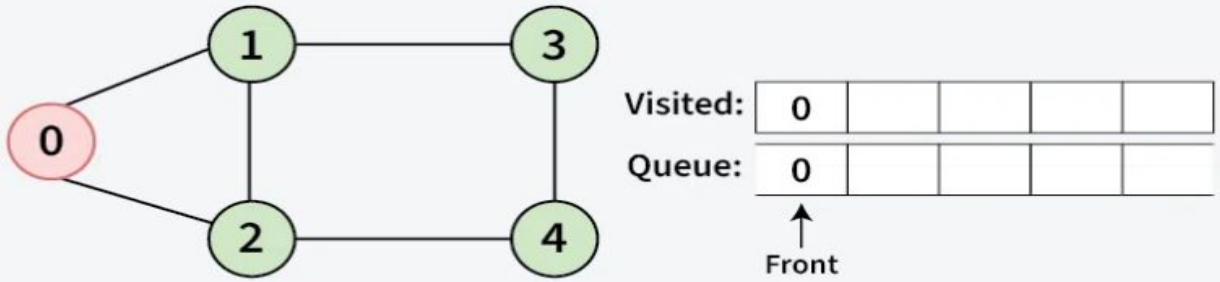
Note: Example solve in class

Breadth-First Search (BFS) Example

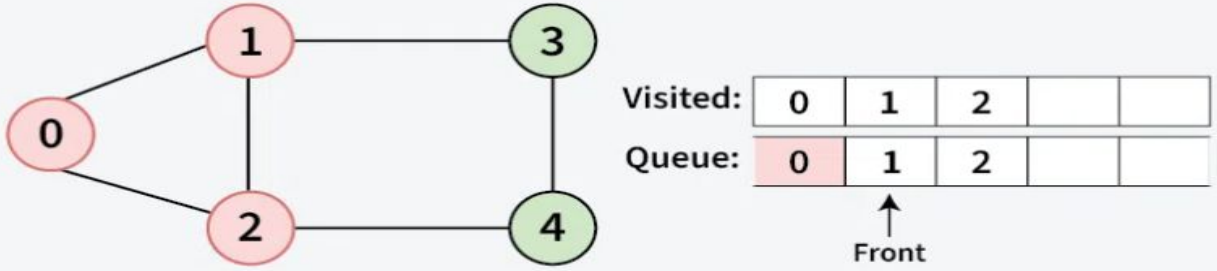
01 Step | Initially queue and visited array are empty.



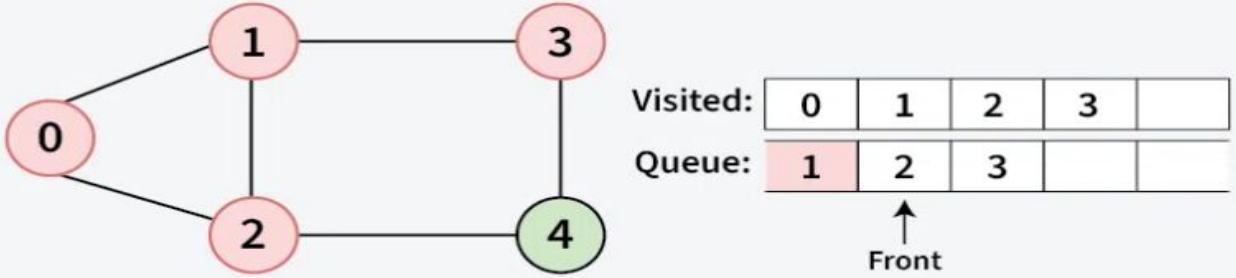
02 Step | Push 0 into queue and mark it visited.



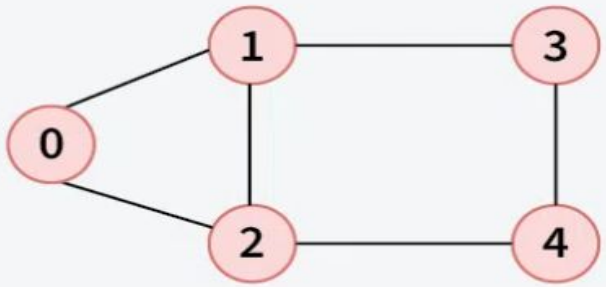
03 Step | Remove 0 from the front of queue and visit the unvisited neighbours and push them into queue.



04 Step | Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



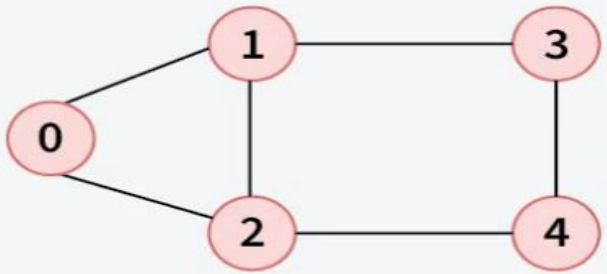
05 Step | Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited:	0	1	2	3	4
Queue:	2	3	4		

Front

06 Step | Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.

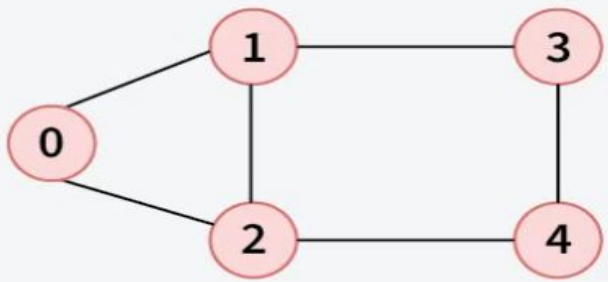


Visited:	0	1	2	3	4
Queue:	3	4			

Front

All neighbors of node 3 have been visited, proceed to the next node in the queue.

07 Step | Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited:	0	1	2	3	4
Queue:	4				

Front

All neighbors of node 4 have been visited, proceed to the next node in the queue.

Breadth-First Search (BFS)

Advantages

- Completeness
- Systematic Exploration
- Useful for Finding Closest Solutions

Disadvantages

- Time Complexity
- High Memory Usage

Depth-First Search (DFS)

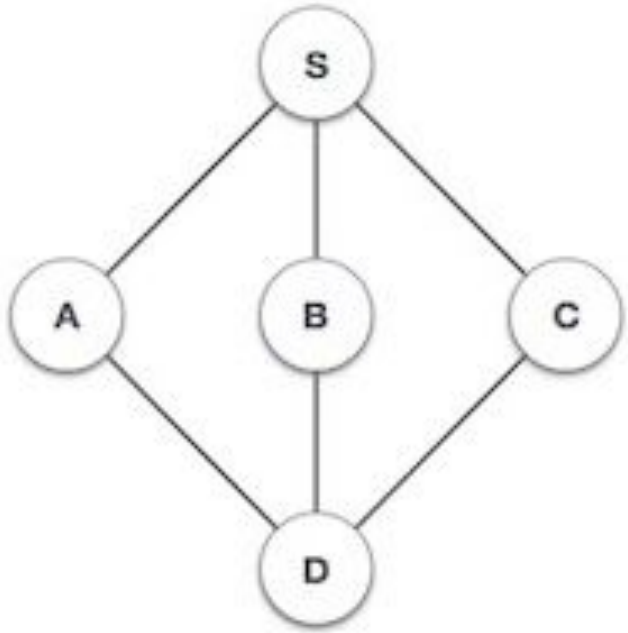
- Depth-First Search (DFS) is a search algorithm commonly used in artificial intelligence (AI) and graph traversal. It explores as far down a branch as possible before backtracking to explore other branches.

Depth-First Search (DFS) Algorithm

In depth-first search the idea is to travel as deep as possible from neighbour to neighbour before backtracking.

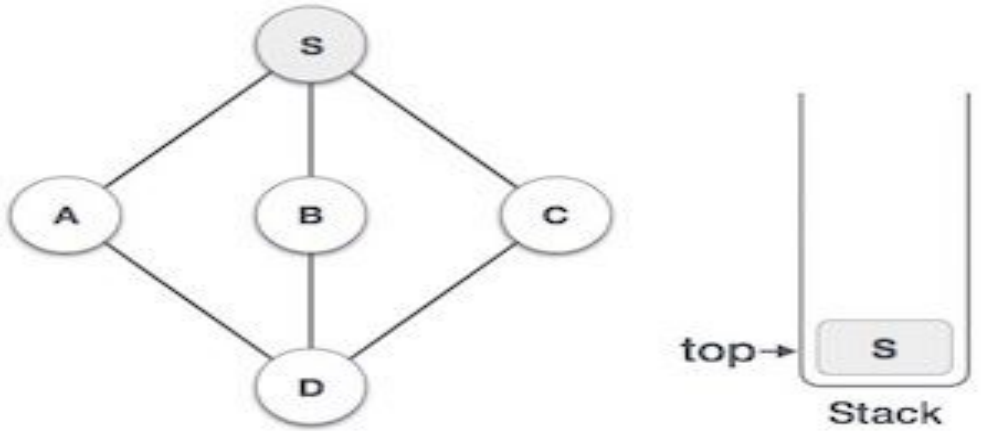
1. Initialize an empty stack and an empty set for visited vertices.
2. Push the start vertex onto the stack.
3. While the stack is not empty:
   - Pop the current vertex.
   - If it's the goal vertex, return "Path found".
   - Add the current vertex to the visited set.
   - Get the neighbours of the current vertex.
   - For each neighbour not visited, push it onto the stack.
4. If the loop completes without finding the goal, return "Path not found“.

Depth-First Search (DFS) Example

1			Initialize the stack.

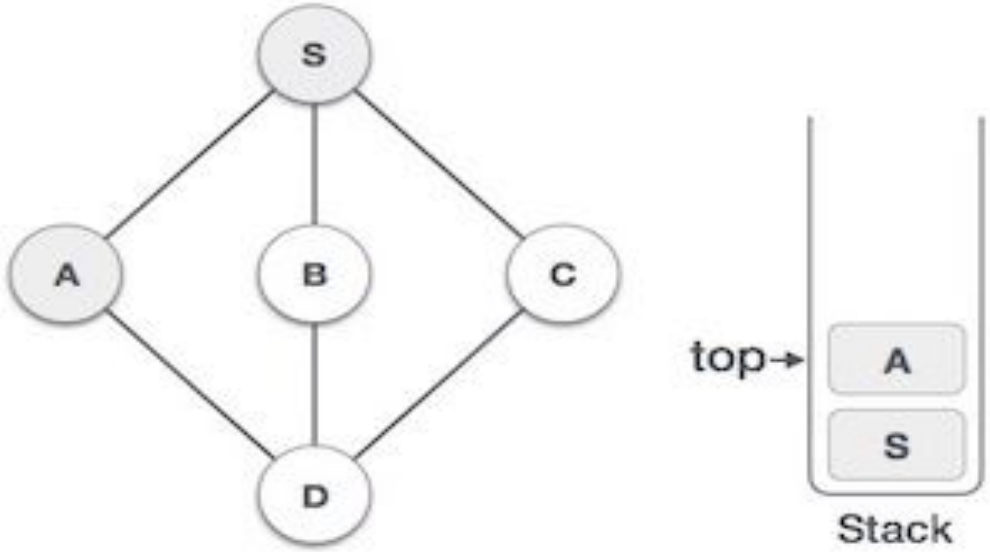
Depth-First Search (DFS) Example

2



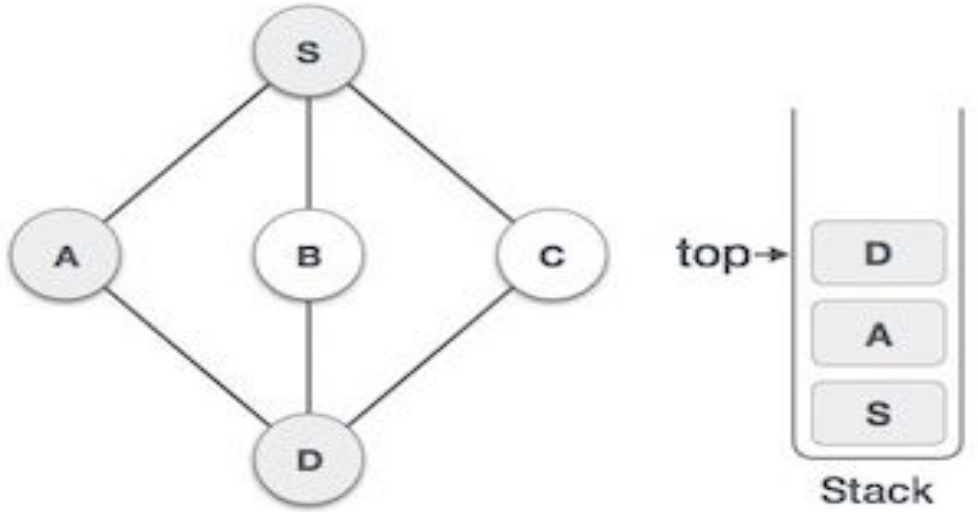
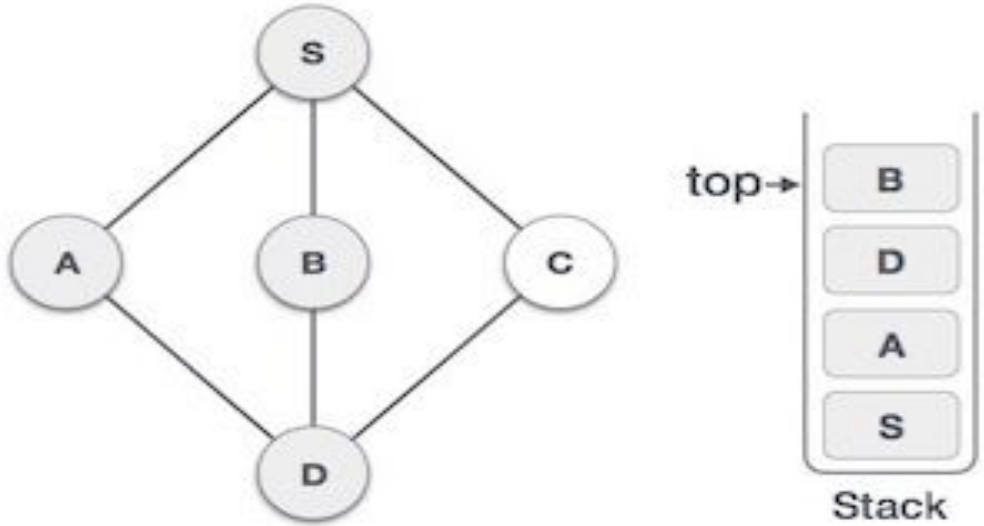
Mark **S** as visited and put it onto the stack. Explore any unvisited adjacent node from **S**. We have three nodes and we can pick any of them. For this example, we shall take the node in an alphabetical order.

3



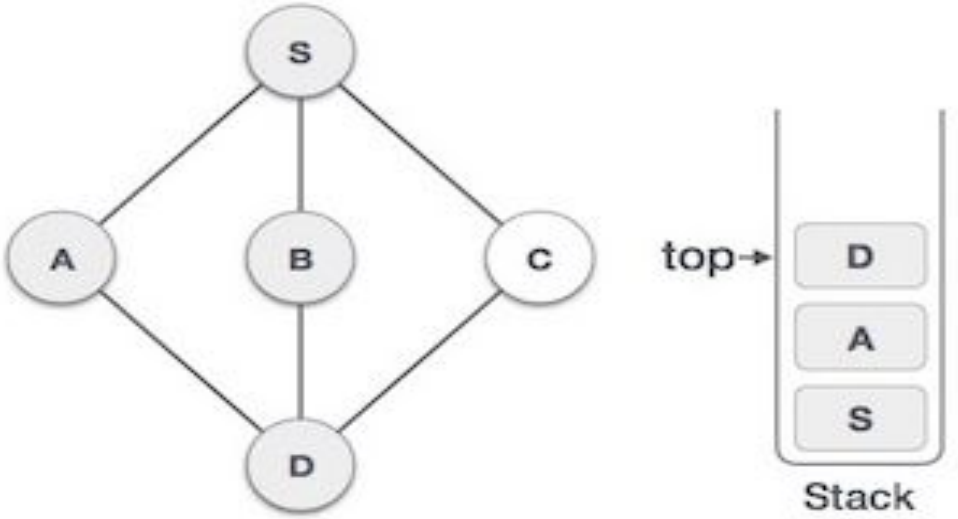
Mark **A** as visited and put it onto the stack. Explore any unvisited adjacent node from A. Both **S** and **D** are adjacent to **A** but we are concerned for unvisited nodes only.

Depth-First Search (DFS) Example

4		Visit **D** and mark it as visited and put onto the stack. Here, we have **B** and **C** nodes, which are adjacent to **D** and both are unvisited. However, we shall again choose in an alphabetical order.
5		We choose **B**, mark it as visited and put onto the stack. Here **B** does not have any unvisited adjacent node. So, we pop **B** from the stack.

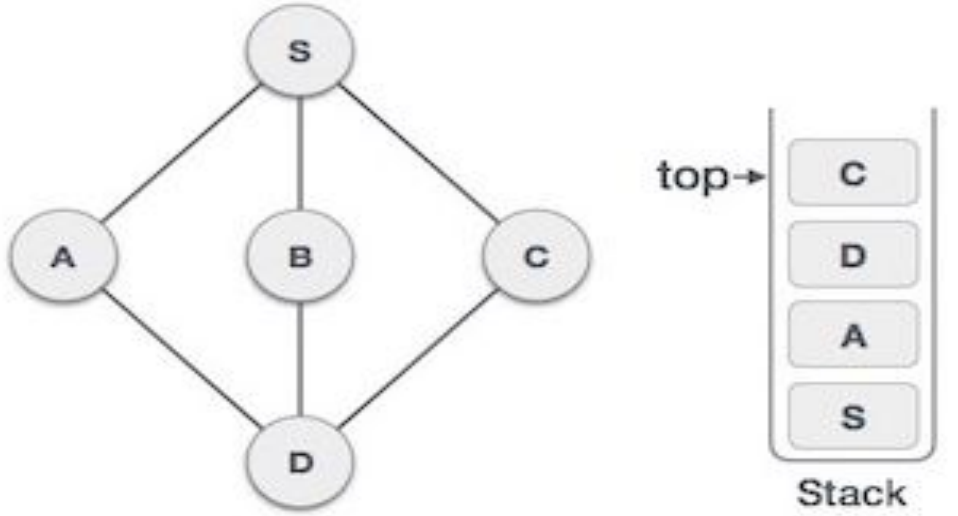
Depth-First Search (DFS) Example

6



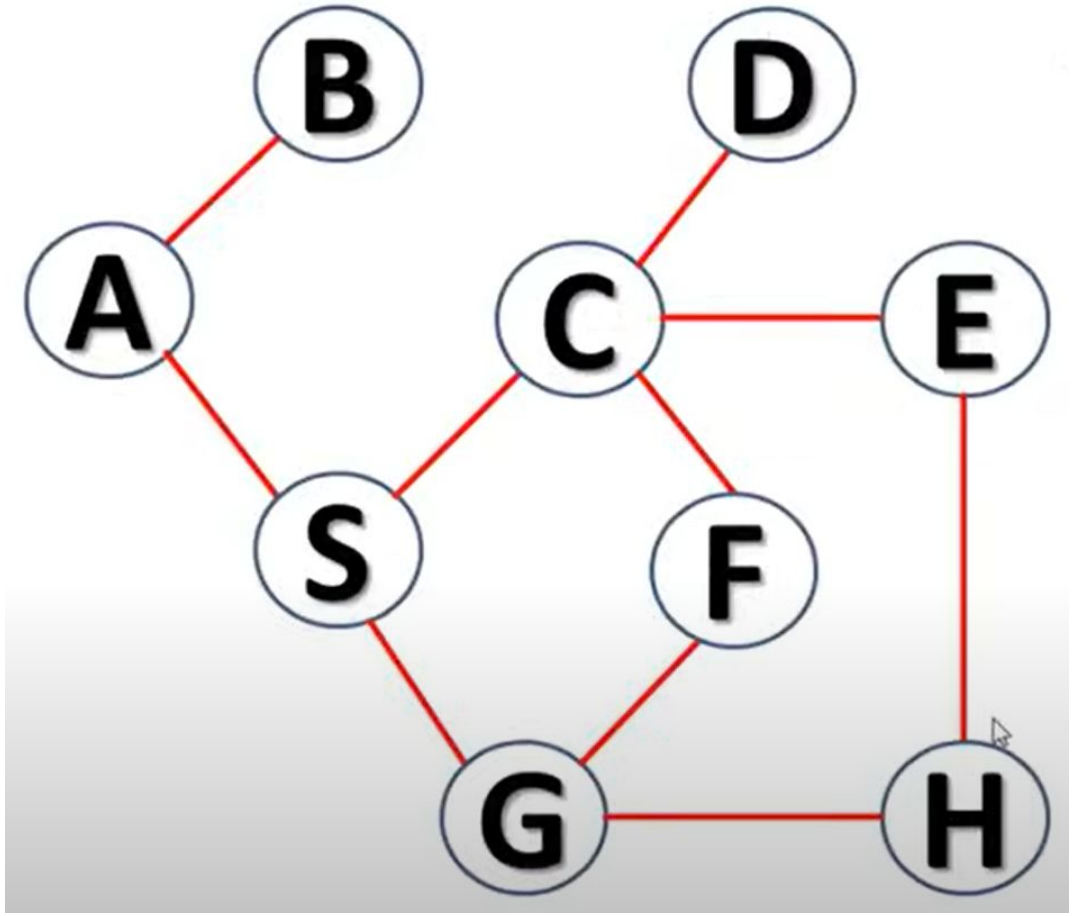
We check the stack top for return to the previous node and check if it has any unvisited nodes. Here, we find **D** to be on the top of the stack.

7



Only unvisited adjacent node is from **D** is **C** now. So we visit **C**, mark it as visited and put it onto the stack.

Depth-First Search (DFS) Example



Depth-First Search (DFS)

Advantages

- Memory Efficiency
- Faster in Finding Feasible Solutions in Deep Trees

Disadvantages

- No Guarantee of Finding the Optimal Solution
- Not Complete in Infinite Search Spaces

Depth Limited Search

- Depth limited search is an uninformed search algorithm which is similar to Depth First Search(DFS). It can be considered equivalent to DFS with a predetermined depth limit 'l'. Nodes at depth l are considered to be nodes without any successors.
- Depth limited search may be thought of as a solution to DFS's infinite path problem; in the Depth limited search algorithm, DFS is run for a finite depth 'l', where 'l' is the depth limit.

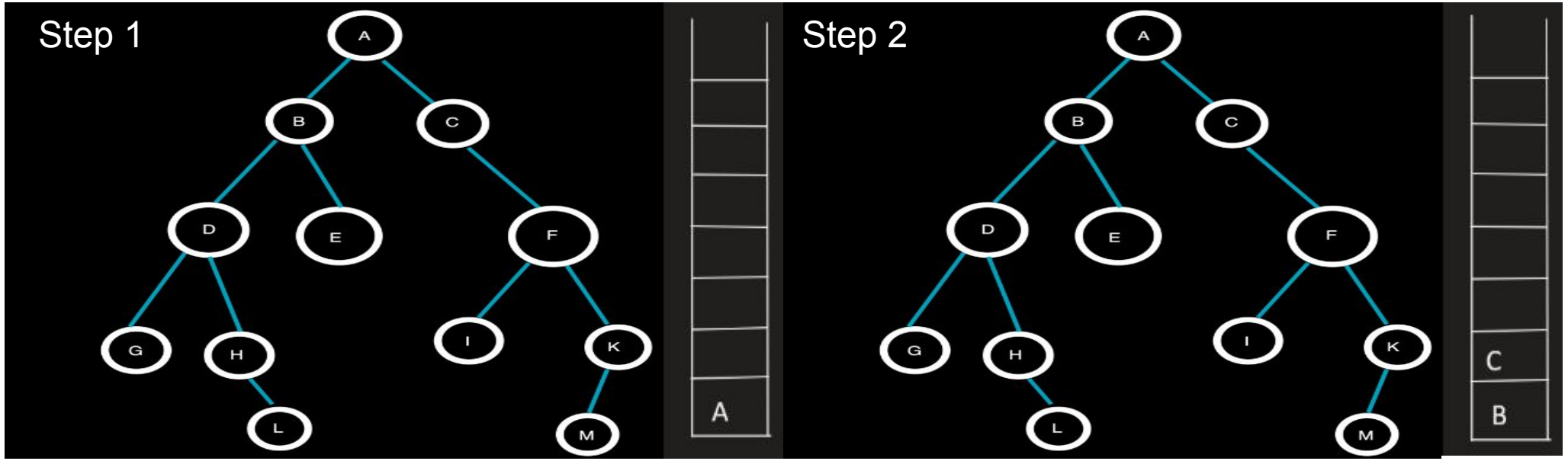
Depth Limited Search Algorithm

We are given a graph G and a depth limit 'l'. Depth Limited Search is carried out in the following way:

- Set STATUS=1(ready) for each of the given nodes in graph G.
- Push the Source node or the Starting node onto the stack and set its STATUS=2(waiting).
- Repeat steps 4 to 5 until the stack is empty or the goal node has been reached.
- Pop the top node T of the stack and set its STATUS=3(visited).
- Push all the neighbours of node T onto the stack in the ready state (STATUS=1) and with a depth less than or equal to depth limit 'l' and set their STATUS=2(waiting). **(END OF LOOP)**
- END

Depth Limited Search Example

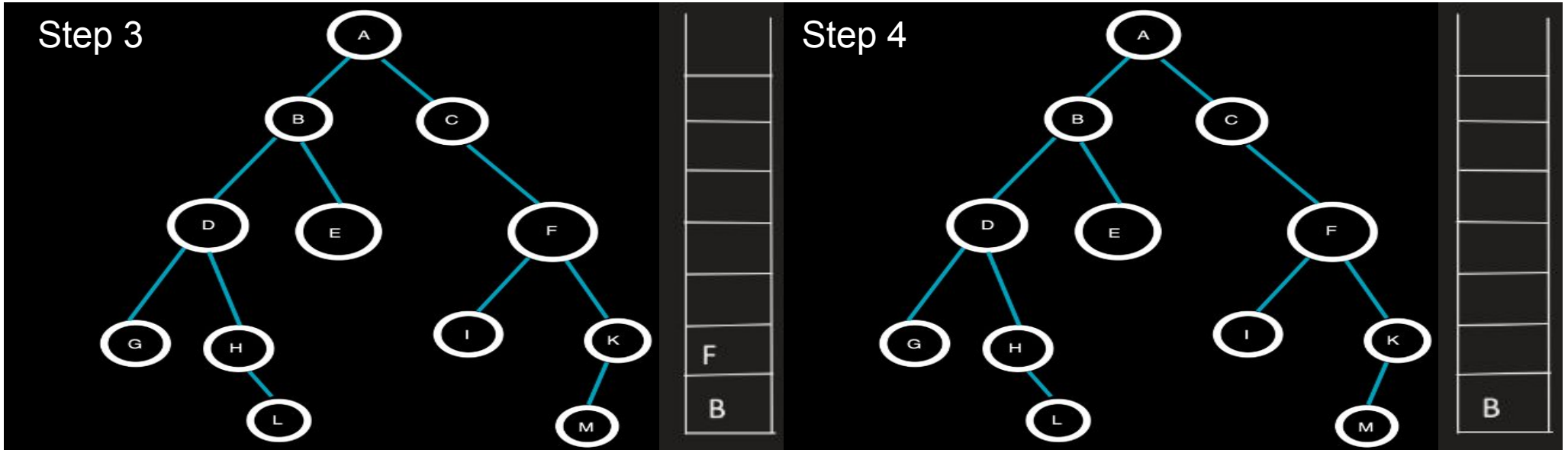
Consider the given graph with Depth Limit(l)=2, Target Node=H and the given source node=A



A being the top element is now popped from the stack and the neighbouring nodes B and C at depth=1(<l) of A are pushed onto the stack.

Traversal: A

Depth Limited Search Example



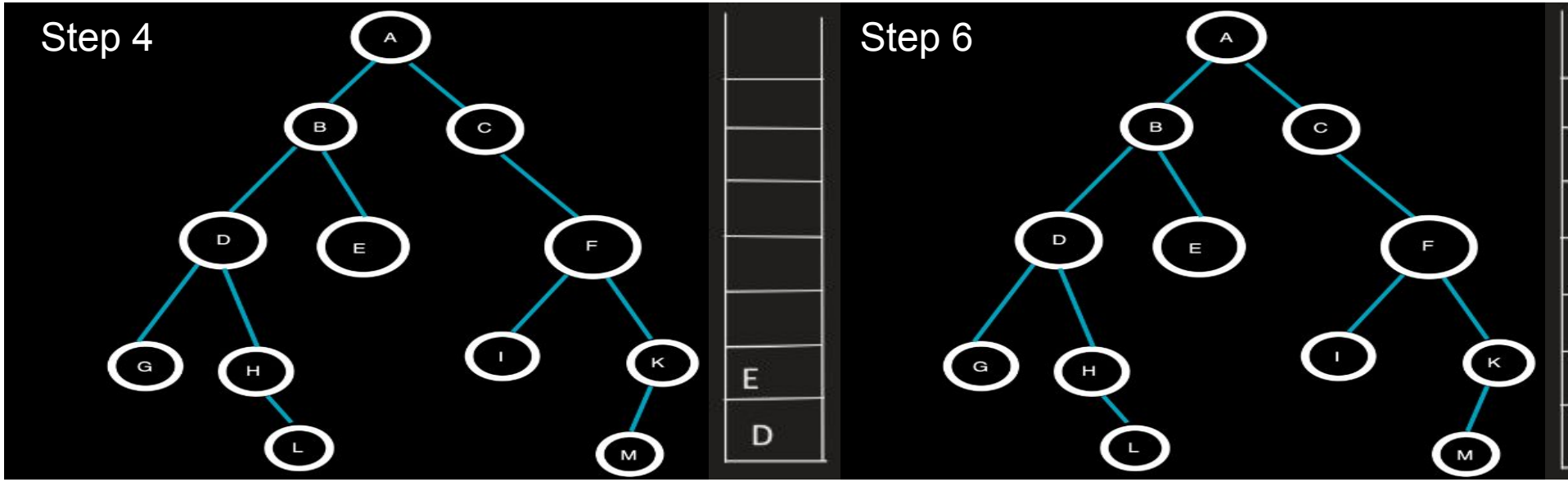
C being the topmost element is popped from the stack and the neighbouring node F at depth=2(=l) is pushed onto the stack.

Traversal: AC

F being the topmost element is popped from the stack and the neighbouring nodes I and K at depth=3(>l) will not be pushed onto the stack.

Traversal: ACF

Depth Limited Search Example



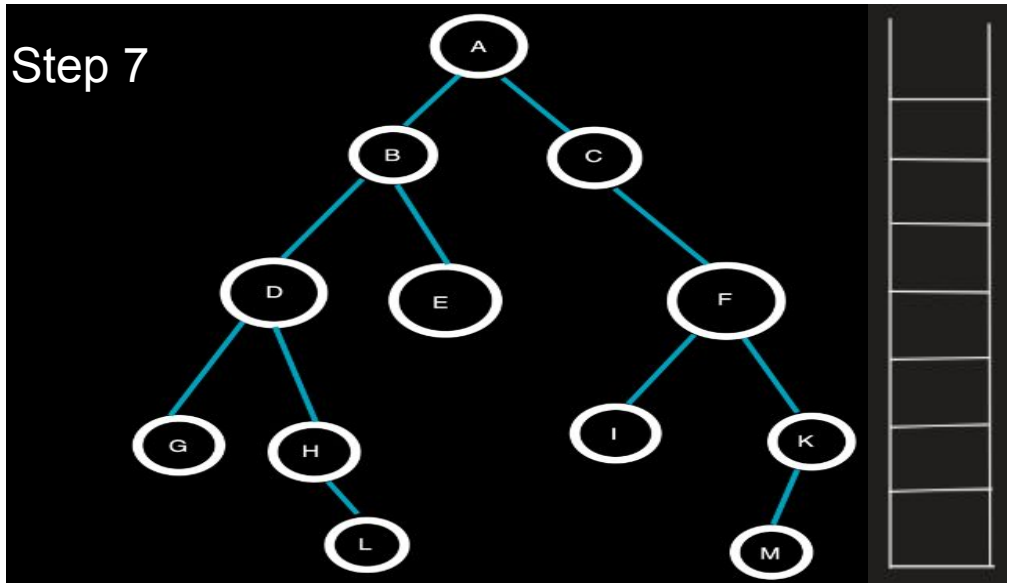
B being the topmost element is popped from the stack and the neighbouring nodes D and E at depth=2(=l) are pushed onto the stack.

Traversal: ACFB

E being the topmost element is popped from the stack and since E has no neighbouring nodes, no nodes are pushed onto the stack.

Traversal: ACFBE

Depth Limited Search Example



D being the topmost element is popped from the stack and the neighbouring nodes G and H at depth=3(>l) will not be pushed onto the stack.

Traversal: ACFBED

Since the stack is now empty, all nodes within the depth limit have been visited, but the target node H has not been reached.

Depth Limited Search (DLS)

Advantages

- Memory Efficiency
- Useful for Large or Infinite Search Spaces
- Controls Depth in Applications with Known Solution Depths

Disadvantages

- Incomplete
- Requires Prior Knowledge of Solution Depth
- Sensitive to Depth Limit Selection

Iterative Deeping Depth First Search

- **Iterative Deepening Depth-First Search (IDDFS)** is a hybrid search algorithm that combines the benefits of Depth-First Search (DFS) and Breadth-First Search (BFS). It systematically increases the depth limit of the search in a depth-first manner, ensuring completeness and optimality similar to BFS while using much less memory.

- Solved example on board

Iterative Deeping Depth First Search Algorithm

1. The Algorithm performs a series of depth-limited DFS searches, starting from a depth limit of 0 and increasing the limit incrementally until the goal is found.
2. For each depth limit L:
   - Perform a DFS, but only expand nodes up to depth L.
   - If the goal is found within this depth, the search stops.
   - Otherwise, increase the depth limit by 1 and repeat.
3. Continue until the goal is found or all nodes are explored.

Iterative Deeping Depth First Search Algorithm (IDDFS)

Advantages

- Memory Efficiency
- Optimal Solution
- IDDFS uses the low memory of DFS and the optimal, complete nature of BFS

Disadvantages

- Repetitive Exploration
- Not Suitable for All Problems where the branching factor is very high